Authentication Vulnerabilities

Authentication vulnerabilities are conceptually simple but disproportionately severe — a flaw here doesn't just leak one piece of data, it hands an attacker the keys to an entire account, and by extension whatever that account can access. This guide ties together threads from across your whole series: your Enum4linux guide's password-policy checks, your Email Enumeration guide's username-harvesting techniques, and your OWASP Top 10:2025 guide's A07:2025 category all converge here.

---

1. Authentication vs. Authorization — Get This Distinction Cold

```
Authentication  → verifying WHO you are ("is this really Carlos123?")
Authorization    → verifying WHAT you're allowed to do ("can Carlos123 delete this acc
ount?")
```

A login form is authentication. Once logged in, whether you're permitted to view another user's order is authorization — exactly the boundary your IDOR guide covered in depth. The two are deeply related (authorization checks are meaningless if authentication itself can be bypassed) but they're distinct failure categories with distinct fixes.

---

2. The Three Factors of Authentication

Factor	Example	Also Called
Something you **know**	Password, security question answer	Knowledge factor
Something you **have**	Phone, hardware security token	Possession factor
Something you **are/do**	Biometrics, behavioral patterns	Inherence factor

True multi-factor authentication combines **different** factor types. Email-based 2FA is a commonly misunderstood example: it looks like two factors, but accessing the email account itself only requires *another password* — so it's really the same knowledge factor verified twice, not genuine 2FA, despite the UX suggesting otherwise.

---

3. How Authentication Vulnerabilities Arise — Two Root Causes

```
1. Weak protection against brute-force attacks
   → the mechanism is theoretically sound, but doesn't stop
     an attacker from simply guessing enough times

2. Logic flaws / poor implementation ("broken authentication")
   → the mechanism can be bypassed entirely through flawed
     reasoning in how it was built, regardless of password strength
```

Because authentication is so foundational to a site's security model, logic bugs here are almost always security-critical — the same class of implementation mistake that might just cause a cosmetic bug elsewhere in the app becomes a full account-takeover primitive when it happens in the login flow.

PART 1: Password-Based Login Vulnerabilities

4. Brute-Force Attacks — Not Just Random Guessing

A brute-force attack uses trial and error to guess valid credentials, usually automated with wordlists. Real attacks aren't blind — they use logic and public knowledge to make **educated** guesses, which is exactly why this connects directly to your OSINT/Email Enumeration guide.

5. Brute-Forcing Usernames

Usernames are often predictable:

```
firstname.lastname@company.com    ← common business login pattern
admin / administrator                ← predictable high-privilege accounts
```

Where to look for username leaks (auditing checklist):

```
- Public user profiles (even if content is hidden, the display name
  sometimes matches the actual login username)
- HTTP responses that accidentally disclose email addresses
  (support tickets, admin contact info)
```

This is precisely the naming-pattern-guessing methodology from your Email Enumeration guide, applied specifically to login credentials rather than phishing

targets.

6. Brute-Forcing Passwords — Exploiting Human Behavior, Not Just Entropy

Password policies (minimum length, mixed case, special characters) make passwords harder for a computer to crack blindly, but users routinely just bend a memorable password to fit the rules:

```
mypassword rejected → Mypassword1!  or  Myp4$$w0rd
```

And when periodic password changes are enforced, users tend to make small, predictable edits:

```
Mypassword1!  →  Mypassword1?  →  Mypassword2!
```

Practical implication: a smart brute-force wordlist targets these *human* patterns specifically, rather than pure random character combinations — far more effective than a naive dictionary attack.

7. Username Enumeration — Finding Valid Usernames via Behavioral Differences

Username enumeration means observing differences in the application's behavior to determine whether a guessed username is valid — this dramatically narrows a brute-force attack, since you can build a confirmed-valid username shortlist before ever guessing a single password.

Three signals to watch for, systematically, on any login/registration form:

a) Status codes
Most wrong guesses should return an identical status code. Any deviation on a specific guess is a strong signal that guess hit something different server-side (a valid username).

b) Error messages
Best practice is an identical, generic message ("invalid username or password") regardless of which part was wrong — but subtle differences (even a single stray character, invisible on the rendered page but present in the raw HTML/response) leak whether the username specifically was valid.

c) Response times
If an app only checks the password *after* confirming the username exists, that extra lookup step can cause a measurably longer response time for valid usernames — and this timing gap can be deliberately amplified by submitting an unusually long password, making the delay far more obvious to detect.

```
# This is exactly where Burp Intruder / a custom script com
es in -
# automate submitting a username wordlist and diff the resp
onses
# for status code, message content, and timing across every
guess
```

8. Flawed Brute-Force Protection

The two standard defenses — **account locking** and **IP-based rate limiting** — both have well-documented, exploitable weaknesses when implemented with flawed logic.

a) Account Locking — Bypassed via Password Spraying
Locking an account after N failed attempts stops a *targeted* attack against one account, but doesn't stop an attacker targeting *many* accounts at once with very few guesses per account:

```
1. Build a list of candidate valid usernames (Section 7)
2. Choose a shortlist of likely passwords — the number must NOT
   exceed the lockout threshold (e.g., if lockout triggers at 3
   attempts, pick at most 3 passwords)
3. Try each password against EVERY username on the list
   (Burp Intruder: "cluster bomb" attack type)
4. You only need ONE user to have used ONE of your guessed
   passwords to compromise an account, and no single account
   ever hits the lockout threshold
```

This is literally the same password-spraying technique your Enum4linux guide flagged as safe/effective specifically **when the password policy shows "Lockout threshold: None"** — that earlier finding and this technique are the same attack, just now explained from the web-app side instead of the SMB side.

Account locking also completely fails to stop credential stuffing — using genuine `username:password` pairs leaked from unrelated breaches (your Email

Enumeration guide's HaveIBeenPwned correlation), since each username is only tried **once** with its already-known real password, never triggering a lockout threshold at all.

b) IP Rate Limiting — Flawed Reset Logic
Some implementations reset the failed-attempt counter whenever the IP owner logs in successfully — meaning an attacker just needs to interject their own valid login every few guesses to keep the counter from ever reaching the block threshold, rendering the defense close to useless.

c) IP Rate Limiting — IP Manipulation Bypass
Since the block is based on the apparent source IP, headers like `X-Forwarded-For` can sometimes be manipulated to make each request appear to originate from a different address, sidestepping the rate limit entirely.

d) Multiple Guesses Per Request
If an application's login endpoint can be coerced into accepting an array/list of password guesses within a *single* HTTP request (a parameter-parsing quirk, not a documented feature), an attacker can test many passwords while only counting as one request against the rate limit.

---

9. HTTP Basic Authentication — An Inherently Weak Mechanism

```
Authorization: Basic base64(username:password)
```

The browser sends this header on **every single request**, not just at login. Several structural weaknesses:

```
- Base64 is ENCODING, not encryption — trivially reversible,
  so without HSTS/TLS the credentials are exposed to a
  man-in-the-middle capture on every request
- Implementations often lack brute-force protection entirely,
  since the "token" is just static encoded credentials with
  no session/rate-limiting layer built in
- No inherent CSRF protection — since credentials are
  automatically resent by the browser, HTTP Basic Auth offers
  nothing to prevent forged cross-site requests riding on it
```

Why a "boring" HTTP Basic Auth page still matters in an assessment: even if the page it protects seems unimportant, credentials captured this way are

frequently reused elsewhere — exactly the credential-reuse risk your Email Enumeration guide's breach-correlation section already covered.

PART 2: Multi-Factor Authentication Vulnerabilities

10. 2FA Token Delivery — Not All "Something You Have" Is Equal

Dedicated hardware tokens / authenticator apps (Google Authenticator, RSA tokens) generate the code directly on the device — nothing is transmitted that could be intercepted in transit.

SMS-based codes are weaker despite still technically being "something you have":

```
- The code is TRANSMITTED, not generated on-device → interceptable
- SIM swapping — an attacker fraudulently obtains a new SIM card
  tied to the victim's phone number, then receives all SMS
  messages (including the 2FA code) intended for the victim
```

11. Bypassing 2FA Entirely — The "Already Logged In" Flaw

If a login flow is genuinely two separate steps (password page, then a *separate* verification-code page), the user is often already in a technically "logged-in" session state after step one, purely by how the session/cookie was issued — before ever completing step two.

Testing methodology: after submitting valid credentials on step one, don't submit the verification code at all — instead, try navigating **directly** to a logged-in-only page/URL. If the application never actually re-checks whether step two was completed, you're in, having never provided the second factor at all.

12. Flawed 2FA Verification Logic — The Cookie-Swap Attack

This is the single most important technique in this whole section — a genuinely devastating logic flaw:

```
Step 1: POST /login-steps/first
        username=carlos&password=qwerty

Server response: Set-Cookie: account=carlos
                (this cookie determines WHICH account step 2 applies to)
```

```
Step 2: POST /login-steps/second
        Cookie: account=carlos
        verification-code=123456
```

The attack: log in with **your own** valid credentials in step one (you legitimately know your own password), but then **change the `account` cookie to a victim's username** before submitting step two:

```
POST /login-steps/second
Cookie: account=victim-user
verification-code=123456
```

If the application only uses the cookie to decide which account's session to grant — and never re-validates that the cookie's username actually matches whoever completed step one — you can now attempt to brute-force the victim's verification code directly, **having never needed to know their password at all.**

Why this is so severe: it completely decouples "knowing the password" from "completing the login" — the attacker's own valid password becomes an all-purpose key to reach the verification step for *any* username, turning 2FA brute-forcing into the *only* remaining barrier.

13. Brute-Forcing 2FA Verification Codes

2FA codes are frequently just 4-6 digit numbers — trivially brute-forceable if not properly rate-limited, exactly the same underlying weakness as Section 8's password brute-force protection, just applied to a numeric code instead of a password.

A common but ineffective defense: automatically logging the user out after too many incorrect code attempts. This looks like protection, but a determined attacker can automate the entire multi-step flow (re-login, resubmit the cookie-swapped account, try the next code) using Burp Intruder macros or the Turbo Intruder extension, defeating the logout-based defense through sheer automation.

PART 3: Other Authentication Mechanisms

14. Keeping Users Logged In ("Remember Me")

Persistent login functionality typically issues a long-lived token/cookie so the user doesn't need to re-authenticate on every visit. If this token is:

```
- Predictable (e.g., a simple encoding of the username, or a
  sequential/guessable value)
- Never expires or rotates
- Not tied to any additional session validation
```

...then it becomes an attractive, long-lived target — stealing or guessing this single token can grant persistent account access without ever touching the password or 2FA flow at all.

15. Resetting Passwords — Two Classic Weak Patterns

a) Sending the actual password by email
If a "forgot password" flow emails the user's **existing plaintext password** back to them (rather than a reset link), this reveals the application is storing passwords in plaintext or reversibly-encrypted form rather than properly hashed — a severe finding on its own regardless of anything else, since it means a database compromise exposes every password directly.

b) Resetting passwords via a predictable/weak URL token

```
https://vulnerable-website.com/reset-password?user=carlos&token=abc123
```

If the reset token is:

```
- Predictable/sequential rather than cryptographically random
- Not properly bound to the specific user it was issued for
- Not single-use (reusable after the reset completes)
- Leaked via the Referer header when the reset page loads
  external resources (images, scripts) - the full URL including
  the token gets sent to that third party
```

...an attacker can potentially guess, reuse, or intercept a token to reset **another user's** password directly — one of the most direct account-takeover paths in this entire guide, since it doesn't even require brute-forcing anything if the token generation itself is weak.

16. Changing User Passwords — Missing Current-Password Confirmation

A change-password feature that doesn't require the user to re-enter their **current** password before setting a new one is dangerous in scenarios like session hijacking or CSRF — an attacker with a stolen session token (but not the actual password) could lock the legitimate user out entirely by silently changing their password, something that wouldn't be possible if current-password re-confirmation were required.

PART 4: OAuth Authentication Vulnerabilities

OAuth introduces its own vulnerability category — briefly worth knowing since it connects directly to your Open Redirect guide's Section 6c chaining example, where a weak `redirect_uri` validation combined with an open redirect elsewhere in the client app enabled stealing a victim's OAuth access token entirely. OAuth-specific flaws are a large enough topic to warrant treatment as its own dedicated guide if you want to go deeper later.

PART 5: Testing Methodology (Putting It All Together)

```
1. Map every authentication-related endpoint: login, registration,
   password reset, 2FA verification, "remember me" issuance,
   password change

2. Test username enumeration on login AND registration forms
   (status codes, error message diffs, response timing)

3. Test brute-force protection:
   - Does account locking exist? Can it be bypassed via
     password spraying (few passwords, many usernames)?
   - Does rate limiting reset on successful login from the
     same IP? Can source IP be spoofed via headers?

4. If 2FA exists:
   - Try skipping straight to logged-in pages after step one
   - Capture the request flow between step one and step two -
     is there a cookie/parameter that determines WHICH account
     step two applies to? Can it be swapped?
   - Test brute-forcing the verification code itself

5. Test password reset flow:
   - Is the token predictable/sequential?
   - Does the reset page load any external resources that
     would leak the token via Referer?
   - Is the token properly invalidated after one use?

6. Test password change flow:
```

```
   - Is current password required to set a new one?

7. If HTTP Basic Auth is present anywhere, confirm HSTS/TLS
   is properly enforced site-wide
```

PART 6: Prevention — Securing Your Own Authentication Mechanisms

```
Passwords:
- Enforce genuinely high-entropy requirements, but recognize
  users will still bend rules predictably (Section 6) - combine
  policy with breach-list checking (reject known-compromised
  passwords, e.g. via the HaveIBeenPwned Pwned Passwords API)
- Use a strong, slow hashing algorithm (bcrypt, Argon2) -
  NEVER store plaintext or reversibly-encrypted passwords

Brute-force protection:
- Use rate limiting over strict account locking where possible
  (locking is more prone to enabling username enumeration and
  denial-of-service abuse against legitimate users)
- Ensure lockout/rate-limit counters DON'T reset simply because
  the source IP logged in successfully once
- Return identical status codes, generic error messages, and
  consistent response timing regardless of which credential
  was wrong

2FA:
- Never let session state advance past step one until step two
  is genuinely verified server-side
- Bind the second-factor verification step to a properly
  validated session identifier tied to the SAME login attempt -
  never trust a client-modifiable value (cookie/parameter) to
  determine which account the second factor applies to
- Rate-limit/lock verification code attempts independently
  from the password step

Password reset:
- Use cryptographically random, single-use, time-limited,
  user-bound tokens
- Never email the actual password
- Avoid loading external resources on the reset page itself

General:
- Require current-password re-confirmation for sensitive
  account changes
- Treat authentication logic with the same "no shortcuts,
  no case-by-case exceptions" discipline emphasized throughout
  your SQLi/XXE/SSTI guides - a single overlooked edge case
  in login logic is disproportionately catastrophic here
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-307 (Improper Restriction of Excessive Authentication Attempts), CWE-287 (Improper Authentication), CWE-620 (Unverified Password Change), CWE-640 (Weak Password Recovery Mechanism)
OWASP Top 10:2025	A07:2025 Authentication Failures
CVSS	Auth bypass/takeover routinely scores High/Critical — PR:N (no privileges required) is common, since the whole point is gaining privileges you don't have
Cyber Kill Chain	Exploitation (bypassing auth) or Weaponization (a credential-stuffing wordlist built from breach data, per your Email Enumeration guide)
MITRE ATT&CK	T1110 (Brute Force) — with sub-techniques T1110.001 (Password Guessing), T1110.003 (Password Spraying), T1110.004 (Credential Stuffing); T1078 (Valid Accounts) once credentials are obtained

Quick Reference Cheat Sheet

```
Username enumeration signals:
Status code diffs | Error message diffs | Response timing diffs

Password spraying (defeats account lockout):
few passwords × many usernames  (never exceed the lockout threshold per user)

2FA bypass techniques:
1. Try navigating directly to logged-in pages after step 1
2. Swap the account-identifying cookie/param before submitting
   the verification code (Section 12 - the big one)
3. Brute-force the verification code itself (usually 4-6 digits)

Password reset red flags:
- Actual password emailed (not a reset link)
- Predictable/sequential reset tokens
- Token not bound to specific user
- Token reusable after use
- Token leaked via Referer header on external resource load
```